Deep Past Challenge – Translate Akkadian to English

> 下面是一份可直接拿去“立项 + 分工 + 开干”的《Deep Past Challenge – Translate Akkadian to English：赛题详细报告 + 参赛导航》（Markdown 版）。我把**【事实】**与**【推断】**严格分开；凡是我目前没法从公开页面稳定抓到的细节，都用 **【待核验】** 标注，并给出“怎么核验”。

《赛题详细报告 + 参赛导航》（团队可执行版）

Assumptions（默认假设，信息不足仍继续写）

- 团队：2 人（均男性），分工未定；每人本地 GPU 显存 **8GB**；可选云 GPU（按需）。
- 目标：**先以学习/工程化演练为主，保留冲奖与可发布产出的上限**（两周内先把 pipeline 打稳）。
- 工具栈：PyTorch / HuggingFace Transformers / PEFT(LoRA) / SentencePiece（你已确认愿意使用）。
- 你们对“机器翻译/NLP”有一定基础但未必做过低资源 MT 的系统冲榜（因此报告更强调“最小闭环 + 可复现 + 防过拟合”）。

---

0. Executive Summary（一页摘要）

赛题一句话定义

把**阿卡德语（Akkadian）楔形文字的学术转写文本（transliteration）**翻译成**英文**；提交每条样本的英文译文；评分为 **BLEU 与 chrF++ 的几何平均**：

$$\text{Score} = \sqrt{\text{BLEU} \times \text{chrF++}}$$

我们的战略：为什么现在做它（以及与 WiDS 的衔接价值）

- Deep Past 截止更早：Kaggle 时间线显示 **Final Submission Deadline = 2026-03-23**（同时存在 Entry/Team Merger 截止点）。
- WiDS Global Datathon 2026 的 Kaggle 页面显示 **Team Merger Deadline = 2026-04-24，Final Submission Deadline = 2026-05-01**。
- 这意味着：先用 Deep Past 把“**数据治理→CV→训练→推理→提交→复现→迭代**”跑顺，再去 WiDS 做更大规模的团队协作与工程化扩展（更稳）。

三条结论（先把坑写在脸上）

1. **最关键难点**：低资源 + 强领域（古代商业文本）+ 转写噪声 + 专名/术语一致性；而且评分把 **“词级片段匹配(BLEU)”与“字符级鲁棒(chrF++)”**绑在一起，单点优化会翻车。
2. **最可能的突破口**：以**“字符/子词鲁棒的 tokenizer + 迁移学习（多语/字节级模型）+ 合规外部平行语料（如 ORACC）”**为主轴，配合严格 GroupKFold 防泄漏。外部数据与预训练模型在规则中被允许（但要守许可与可复现）。
3. **最容易踩的坑**：
   - 公榜只用约 **34% 测试集**，很容易对公榜过拟合。
   - 盲目用 LLM 后处理“润色”可能**提升可读性但降低分数**（因为 n-gram/字符匹配被破坏）。
   - 数据中可能存在拼写/符号不一致（有人报告 test.csv 有 train 没有的拼写问题），需要做鲁棒归一化与诊断。

---

1. 赛题规则与事实核验（只写【事实】，每条给来源）

> 说明：Kaggle 页面部分内容在一些抓取环境里会“不可展开”，但关键字段可从公开条目/时间线/规则摘要中获得。凡缺失我都标【待核验】并告诉你到 Kaggle 哪个 Tab 看。

1.1 截止日期 / 阶段安排（【事实】）

- Deep Past（Kaggle 时间线摘要）：
  - Start Date：**2025-12-16**
  - Entry Deadline：**2026-03-16**
  - Final Submission Deadline：**2026-03-23**
  - Team Merger Deadline：时间线中存在该项，但我当前抓取摘要未稳定显示具体日期 **【待核验】**（通常与 Entry Deadline 同日或接近；请在 Kaggle Competition → *Timeline* 栏确认）。
- WiDS Global Datathon 2026（用于你们整体排期对齐）：
  - Team Merger Deadline：**2026-04-24**
  - Final Submission Deadline：**2026-05-01**

1.2 评估指标（【事实】）

- 评分是 **BLEU 与 chrF++ 的几何平均**：BLEU×chrF++\sqrt{BLEU \times chrF++}BLEU×chrF++。
- chrF++ 的原始定义来自 WMT 论文（字符 n-gram F-score + 词 n-gram 增强）。
- BLEU 的经典定义来自 Papineni et al. 2002。

1.3 数据形式（【事实】）

- Kaggle Data 页摘要：输入为 Akkadian **transliteration（拉丁字母转写）**，目标为英文翻译；赛题面向“数千条”楔形文字文本。
- 你提供的“清单原文”与此一致：输入是 transliteration，不是楔形文字图片/Unicode 楔形文字本体。（你们的现有理解与官方摘要一致。）

1.4 外部数据 / 预训练模型限制（【事实】）

- Kaggle Rules 摘要明确：**允许使用外部数据源与预训练模型**（规则原文在 Rules 页）。
- 因此：使用 ORACC 等公开语料在“规则层面”可行，但仍需逐一核查数据许可与引用要求（合规层面属于你们自己的风控）。

1.5 提交格式 / 提交限制（【事实】）

- Rules 摘要包含：提交文件需命名为 **submission.csv**。
- 某公开 Notebook/代码片段显示 submission DataFrame 列为 `['id','translation']`（用于对齐格式）。
- Submissions per Day：**每天最多 5 次提交**。
- Final scoring 选择条数：规则摘要出现“可选择最多 2 个提交用于最终评分”的表述（需在 Rules 页核对完整上下文）**【待核验】**。

1.6 推理时间 / 显存限制 / 离线在线（【待核验】）

- 该赛题被标注为 **“Code Competition”**（模型页/赛题页导航含 Code/Models 等），通常意味着需要在 Kaggle Notebook 环境离线推理出提交文件。
- 但具体 **Notebook 运行时长、GPU/CPU 可用时长、RAM、磁盘、是否允许联网** 等，需要在 Kaggle Competition → *Code Requirements* 或 *Rules* 的详细条款里确认。当前抓取未拿到原文细节，故标 **【待核验】**。
  - 核验路径：Kaggle 赛题页 → Code →（右侧）Requirements / 或 Rules 全文。

---

2. 具体难点（攻关清单风格）

2.1 为什么这题难（按机制拆解）

- **低资源机器翻译**：有效平行语料量有限、文本体裁集中（商业/书信/账目）、术语密集；泛化主要靠迁移学习与数据工程。
- **Akkadian 转写的“噪声形态”**：同一符号/词可能有多种转写习惯；夹杂断裂/缺损标记、括号、数字、连字符等；模型若只学表面 token，会在 test 分布漂移时崩。
- **指标绑架策略**：BLEU 更偏词/短语 n-gram，chrF++ 更偏字符级相似；你想“润色英文”可能提升语义但降低分数（n-gram 被改写）。
- **比赛工程难**：

- CV 若按行随机切分，可能把同一泥板/同一文书来源泄漏到 train/val，导致本地高分、公榜/私榜崩。
- 公榜只用约 34% 测试集，调参很容易过拟合公榜。

2.2 难点诊断表（必须“可观测、可排查、可修复”）

难点	可观测症状	排查方法（证明它在影响分数）	修复策略（可落地）
领域偏移/低资源	val 高、公榜低；或 seed 波动巨大	1) GroupKFold vs RandomKFold 对比；2) 画 BLEU/chrf++ 分项曲线	迁移学习（mT5/mBART/ByT5 等）+ 继续预训练(TAPT/DAPT 思路)
专名/术语不一致（神名/地名/度量衡）	输出多样化、同一实体多种拼写	统计专名 token 的一致性；对同 id 的相似句抽检	建术语表 + 约束解码（lexically constrained decoding）
转写噪声（连字符/缺损符号/变体）	chrF++ 低；错在少数字符	chrF++ 错误热力图；对比“归一化前后”分数	统一归一化（NFKC、标点/连字符策略）、噪声增强训练
tokenizer 不适配（OOV/碎片化严…	BLEU 低且 rare-word 错误集中	统计子词覆盖率、平均 token 长度；专名被切碎	SentencePiece(unigram/BPE)对比；或直接 ByT5 字节级
beam search 反常	beam 越大分越低	固定模型，对比 num_beams=1/4/8；看长度分布	调 length penalty；或用蒸馏/训练目标匹配推理策略
公榜过拟合	公榜涨、本地不涨；换子集评估立刻崩	用“本地模拟公榜子集”/多折外推；记录每次提交改动	冻结提交频率；用“阈值规则”决定是否提交；只提交系统性改进
盲目 LLM 后处理	可读性↑但分数↓	对同一预测：后处理前后 BLEU/chrf++ 变化	只做“格式化不改写”的后处理（空格/标点/大小写）

---

3. 需要的先验知识（最小集合 + 怎么用）

A) 入门必备（1–2 天补齐）

1. **Seq2Seq/Transformer 基础**：encoder-decoder、teacher forcing、最大似然训练。
   - 用法：直接套 HF `AutoModelForSeq2SeqLM` 微调 baseline。
2. **Tokenization**：BPE vs Unigram（SentencePiece）。
   - 用法：做 2 套 tokenizer 对照（同模型、同训练步数）看 BLEU/chrf++ 分项。
3. **指标理解**：BLEU（n-gram + brevity penalty）、chrF++（字符级 F-score）。
   - 用法：训练日志里必须拆分记录 BLEU 与 chrF++，避免“总分上升但其中一项塌陷”。

B) 核心壁垒（决定能否上分）

1. **低资源 MT：回译/数据增强/迁移学习**
   - 用法：引入合规外部平行语料（如 ORACC 平行对），并做“来源分层 CV”验证泛化。
2. **稳健验证：GroupKFold 防泄漏**
   - 用法：以“文本来源/泥板ID/项目ID”等为 group（具体字段需你们读 train.csv 头部确认）【待核验】。
3. **训练工程（8GB 现实）**：混合精度、梯度累积、gradient checkpoint、LoRA。
   - 用法：mT5-small / ByT5-small 这类模型用 LoRA 能在 8GB 上跑出可用迭代速度。

C) 冲榜技巧（决定上限）

1. **术语一致性与约束解码**（尤其专名/量词/日期）
   - 用法：构建术语表（从 train 与外部语料抽取），推理时做 lexically constrained decoding 或后处理一致化。
2. **域自适应继续预训练（TAPT/DAPT 思路）**
   - 用法：仅用“未标注的转写文本”(train+test 的 source 侧）做 masked/denoise 式继续预训练，然后再监督微调。
3. **集成与多样性**：不同 tokenizer/seed/架构的加权融合（而不是同质模型堆数量）。
   - 用法：用 OOF 相关性挑“互补模型”再 blend。

---

4. 相关领域与必读论文/资源导航（联网 + 可追溯）

> 你们的阅读目标不是“学完 MT”，而是：**每读一项，就能在 pipeline 里对应一个可验证改动**。

4.1 领域映射（这题到底沾了哪些学科）

- 低资源机器翻译、历史语言学/计算语言学、噪声鲁棒建模、tokenization、领域自适应、术语一致性与约束解码、评估与错误分析。

4.2 资源清单（16 条，按优先级）

> 格式：解决什么 → 读哪里 → 对应 pipeline 环节

1. **Kaggle Metric：DPI BLEU/chrF++** → 读公式与实现细节 → *评估/日志*
2. **chrF++ 原论文（Popović 2017）** → 读指标定义与为何对形态丰富语言更友好 → *错误分析*
3. **BLEU 原论文（Papineni 2002）** → 读 BLEU 组成与 BP → *推理长度控制*
4. **SentencePiece（Kudo 2018）** → 读 Unigram LM tokenizer 直觉与训练 → *tokenizer*
5. **BPE subword（Sennrich 2016）** → 读 rare word 与子词切分 → *tokenizer/稀有词*
6. **ByT5（字节级 seq2seq）** → 读“token-free/byte-level”动机 → *对转写噪声鲁棒*
7. **mT5（multilingual text-to-text）** → 读预训练目标与多语迁移 → *baseline 模型*
8. **mBART（多语去噪预训练）** → 读 denoising pretrain → *baseline/迁移*
9. **NLLB（No Language Left Behind）** → 读低资源覆盖与训练配方 → *迁移学习路线参考*
10. **LoRA（低秩适配）** → 读参数高效微调 → *8GB 训练工程*
11. **Sequence-level KD（Kim & Rush 2016）** → 读如何把大模型知识蒸到小模型、改善推理/beam 需求 → *加速/稳定推理*
12. **Domain Adaptive Pretraining（Gururangan 2020）** → 读 DAPT/TAPT 框架 → *域自适应*
13. **Lexically Constrained Decoding（Post & Vilar 2018）** → 读约束 beam 的基本算法 → *术语一致性*
14. **ORACC Akkadian 资源说明** → 读 Akkadian 注释/转写规范 → *数据清洗与归一化*
15. **ORACC Akkadian-English Parallel Corpus（Kaggle 数据集）** → 看数据字段与许可说明 → *外部数据扩充*
16. **Kaggle 讨论：LLM 后处理可能降分** → 读经验教训 → *后处理策略*

4.3 阅读计划（倒排到截止日）

- **Day 1–2（必须完成）**：#1 #2 #4 #6 + Kaggle 赛题 Data/Rules 快速扫一遍（确定字段与限制）。
- **Week 1（把 baseline 做到“可复现可迭代”）**：#7/#8（二选一主力）+ #10 + #12（理解 TAPT/DAPT 是否值得做）。
- **Week 2（冲分组件）**：#13（术语约束）+ #11（蒸馏/推理稳定）+ #15（外部数据对齐）。

---

5. 模型理论设计 & 工程化实验：难点与攻破路线

5.1 Baseline 方案（48 小时内可跑通并提交）

数据处理（tokenization 方案）

- 主推：**SentencePiece Unigram**（对噪声与稀有词更稳）+ 备选：BPE。
- 第二备选：**ByT5（字节级）**，专治转写里各种奇怪符号/拼写变体，代价是训练更慢、序列更长。

模型（先核验规则后选择）

- 规则允许预训练模型（Rules 摘要）→ 可以用：mT5 / mBART / ByT5 / NLLB 等。
- 8GB 现实优先级（【推断】）：
  1. `mt5-small` + LoRA
  2. `byt5-small` + LoRA
  3. `mbart-large-50`（若显存吃紧就不上）
- 注意：你们也可以直接参考 Kaggle Models 页已有的公开模型与分数作为 sanity check（例如 best public score 显示过 32.6）。

训练与验证（CV 方式）

- 首选：**GroupKFold**（按“文本来源/泥板ID/项目ID”等 group）避免泄漏。字段名需要从 train.csv 头部确认 **【待核验】**。
- 日志必须记录：fold、seed、BLEU、chrF++、总分（几何平均）、预测长度统计（均值/分位数）。

推理（解码与后处理）

- 先用 beam search（beam=4）+ length penalty（1.0 附近），再做网格搜索；不要迷信 beam 越大越好（有人报告 beam 会伤 CV）。
- 后处理只做**不改变用词**的格式修复（空格/重复标点/不可见字符），避免“润色改写”。

最小可复现 repo 结构（建议）

```
deep-past-mt/ README.md requirements.txt configs/ baseline_mt5_lora.yaml
baseline_byt5_lora.yaml data/ raw/ (Kaggle input mount) interim/ processed/ src/
preprocess.py train.py evaluate.py infer.py metrics.py (调用 Kaggle metric 或复刻实现)
utils/ seed.py logging.py scripts/ run_train.sh run_infer.sh outputs/ runs/{run_id}/
checkpoints/ oof/ preds/ logs.jsonl
```

训练命令草案（示例）

- 本地/云训练：

```
python src/train.py --config configs/baseline_mt5_lora.yaml \ --fold 0 --seed 42 --fp16 --grad_accum 8
```

- Kaggle 推理生成提交：

```
python src/infer.py --ckpt outputs/runs/<run_id>/checkpoints/best.pt \ --out submission.csv
```

日志字段规范（强制）

- `run_id, git_commit, model_name, tokenizer, spm_vocab, max_src_len, max_tgt_len`
- `fold, seed, lr, batch_size, grad_accum, epochs, num_beams, length_penalty`
- `val_bleu, val_chrfpp, val_score_geom, train_loss, val_loss`
- `pred_len_mean, pred_len_p90, oov_rate, spm_unk_rate`

5.2 提升路线（Ablation Ladder：12–18 项，按收益/成本/风险排序）

> 评分建议：收益(1-5) / 成本(1-5) / 风险(1-5, 越高越危险)

优先级	改动点	预期收益	成本	风险	必要对照实验	失败信号（止损）
1	统一归一化（连字符/空格/Unicode NFKC）	4	1	1	归一化前后：BLEU/chrf++ 分项	BLEU↑但 chrF++↓明显或反之
2	tokenizer 对照：Unigram vs BPE	4	2	1	同模型同步数，比较分项	分数差 <0.1 且训练更慢
3	ByT5 vs mT5（谁更抗噪）	4	3	2	固定 LoRA 配置对比	ByT5 慢但分不涨
4	LoRA 超参扫（rank/alpha/target modules）	3	2	2	rank=8/16/32	波动极大且无稳定提升
5	合规外部数据：ORACC 平行对混训	5	3	3	只加外部 vs 不加；来源分层 CV	本地涨、公榜不涨（域偏移）
6	噪声增强：随机删/换分隔符、拼写扰动	3	2	2	有/无增强，chrF++ 变化	chrF++ 不动但 BLEU 掉
7	TAPT：用 source 侧无标注继续预训练	4	4	3	TAPT vs no-TAPT	训练不稳定或收益 <0.2

优先级	改动点	预期收益	成本	风险	必要对照实验	失败信号（止损）
8	术语表抽取（专名/量词）+ 一致性后处理	3	2	3	启用/禁用术语策略	BLEU 上升但 chrF++ 下降（改写过多）
9	约束解码（lexically constrained decoding）	4	4	4	术语集大小 k=50/200	推理超时/实现复杂收益小
10	蒸馏：teacher→student（提速稳分）	3	4	3	KD vs no-KD	学生分数掉太多
11	训练技巧：label smoothing / lr schedule	2	2	2	smoothing=0/0.1	BLEU/chrf++ 都不动
12	集成：不同 tokenizer + 不同架构加权	4	3	2	单模 vs 双模/三模	OOF 相关性太高，融合无增益
13	长度建模：最优 length penalty 网格	2	1	1	lp=0.6~1.4	分数不敏感
14	数据清洗：修复 test 拼写变体映射表	3	2	2	修复前后对比	只对极少样本有效
15	提交策略：减少公榜调参	3	1	1	提交前先做“模拟公榜子集”	公榜涨但本地不涨

2 周实验排期表（按天）

- D1：跑通数据读取 + baseline tokenizer（Unigram）+ 1 折训练出首个提交
- D2：补齐 5 折 GroupKFold（字段核验）+ 统一日志规范
- D3：BPE tokenizer 对照实验
- D4：ByT5 小模型对照
- D5：LoRA 超参小扫（rank/alpha）
- D6：归一化/清洗 ablation（强制做）
- D7：噪声增强 ablation（先轻量）
- D8：引入 ORACC 平行对（只混少量，先看方向）
- D9：外部数据比例 sweep（10%/30%/50%）
- D10：TAPT（若算力允许，否则跳过）
- D11：术语表抽取 + 一致性后处理
- D12：融合（挑 2 个互补模型）
- D13：推理超参（beam/lp）固定在最优附近
- D14：冻结方案，做一次“冲刺提交”（最多 1–2 次，避免公榜过拟合）

---

6. 拿奖门槛与 publish 价值评估（框架 + 可核验信息）

6.1 Kaggle 冲奖判断（【事实】+【推断】）

- 【事实】Leaderboard 页摘要显示：公榜基于约 **34% 测试集**；顶部队伍分数在 **38.1** 左右（至少前两名）。
- 【推断】由于评分是 BLEU×chrF++ 的几何平均，且数据低资源，**头部差距可能很小**，想进 Top 10% 通常需要“系统性提升”（外部数据/迁移学习/鲁棒清洗/集成至少占两项）。
- 不确定性来源：私榜分布未知 + 公榜子集偏差（过拟合风险被多次讨论）。

> 建议你们把目标拆成三档（更可执行）：

- **档 A（学习闭环）**：稳定可复现，5 折 OOF 与公榜一致性合理（不过拟合）。
- **档 B（冲分可见）**：单模达到 Kaggle Models 页公开模型量级（比如 30+），再尝试融合。
- **档 C（冲奖）**：外部语料合规引入 + 强 CV + 术语一致性 + 融合（至少 3 件套）。

6.2 Publish 价值：什么情况下值得写博客/短论文？

满足任一条就值得写：

- 你们提出一套“古语言转写→翻译”的**鲁棒 tokenizer/归一化规范**，在多个设置下稳定提升 chrF++ 且不伤 BLEU。
- 你们做了“术语一致性约束/后处理”在不改变句法润色的前提下提升几何平均。
- 你们给出“公榜过拟合诊断框架”（如子集模拟、提交策略、OOF 相关性）并在讨论中可复现。

可发表贡献点候选（3–5 个）+ 最小证据链

1. **鲁棒归一化 + tokenizer 配方**
   - 证据链：Unigram/BPE/ByT5 三角对照 + 归一化消融 + 分项指标提升（BLEU/chrf++）。
2. **术语一致性约束（专名/量词）**
   - 证据链：术语表构建方法 + 约束解码/轻量一致化后处理对照 + 失败案例分析。
3. **外部语料（ORACC）引入的“域偏移控制”**
   - 证据链：外部数据比例 sweep + 按来源分层 CV + 公榜/私榜一致性讨论。
4. **8GB 友好的低资源 MT 工程模板**（LoRA + 复现日志）
   - 证据链：同等预算下 LoRA vs full finetune（若可做）+ 训练成本/速度/分数三维对比。

6.3 风险登记表（触发条件 + 预案）

风险	触发条件	影响	预案
规则风险	使用了不允许的数据/工具	取消资格	每次引入外部资源都做“许可与引用”记录；对照 Rules 原文
数据许可风险	外部语料许可证不清晰	不能公开复现/发布	只用许可清晰或可引用的数据集（ORACC/Kaggle 数据集页面说明）
复现风险	notebook/环境不可复现	难以迭代	固化 requirements + seed + config；输出 artifacts
算力风险	8GB 跑不动大模型	进度崩	以 mT5-small/ByT5-small + LoRA 为主；必要时短租云 GPU
过拟合风险	公榜涨、OOF 不涨	私榜翻车	限制提交频率；以 OOF 为主；记录每次提交差异

---

7. 下一步 ToDo（15–20 个可执行任务，带分工/耗时/产出）

> 先默认分工：

- **A（Research Lead）**：数据诊断/指标分析/实验设计与消融记录
- **B（ML Engineer Lead）**：训练框架/推理脚本/复现与提交流程
  （你们之后可互换）

优先级	任务	分工	耗时（粗略）	产出物
1	打开 Kaggle Data，确认 train/test/sample_submission 字段与行数	A+B	0.5h	`data_schema.md` （字段表）
2	核验 Rules 全文：外部数据、预训练、联网、运行限制	A	0.5h	`rules_checklist.md` （逐条引用）
3	写 `preprocess.py` ：基础清洗 + 可配置归一化开关	B	2h	可复用脚本 + 单测样例
4	搭 5 折 GroupKFold（group 字段待核验）	A	1h	`folds.csv`
5	跑通 mT5-small + LoRA baseline（1 折）	B	6–10h	首个 ckpt + 首次提交
6	接入官方 metric（BLEU/chrf++/几何平均）到本地评估	B	1–2h	`metrics.py` + `eval.json`
7	建立日志规范与 run_id 目录结构	B	1h	`logs.jsonl` 样例
8	做 Unigram vs BPE tokenizer 对照（固定模型与步数）	A	6–10h	对照表 + 结论
9	做 ByT5-small baseline（看抗噪收益）	B	8–12h	分数对照

优先级	任务	分工	耗时（粗略）	产出物
10	实现“长度统计 + 错误分析报告”(top errors/rare words)	A	2–3h	`error_report.md`
11	引入轻量噪声增强（符号扰动/空格/连字符）消融	A	4–6h	ablation 结果
12	检索并下载合规 ORACC 平行语料（或 Kaggle ORACC 数据集）	A	1–2h	`external_data_manifest.md`
13	外部数据混训（10%/30%/50%）+ 分层 CV	B	1–2 天	曲线 + 最优比例
14	术语表抽取（专名/量词）与一致性规则（不改写）	A	4–6h	`term_lexicon.tsv`
15	约束解码 PoC（仅对少量高频术语）	B	1 天	可开关的 constrained decoding
16	融合（2–3 个互补模型）与权重搜索	A+B	0.5–1 天	`blend.py` + 最佳权重
17	制定提交策略（何时提交、提交记录模板）	A	0.5h	`submission_log.md`
18	固化 Kaggle 推理 notebook（离线、可一键产出 submission.csv）	B	2–4h	`kaggle_infer.ipynb`
19	写“可发布技术博客大纲”(若两周内达到档 B/C)	A	2h	`blog_outline.md`
20	每日 10 分钟复盘：记录“做了什么/学到了什么/下一步”	A+B	0.2h/天	`daily_notes/`

你们现在立刻可以怎么用这份报告（最省脑）

- 直接按 **5.1 Baseline** 把 repo 搭起来，48 小时内先出首个提交（哪怕分低，也要闭环）。
- 然后按 **5.2 Ablation Ladder** 从优先级 1→5 往下爬；**每次只改一个变量**，否则你们会在噪声里迷路。
- 全程把 **BLEU 与 chrF++ 分项**当“仪表盘”，别只盯总分。